



# CS-229

Greg Ver Steeg

This week:

Finish unsupervised learning

- A few missing pieces on Flow Matching
- GAN
- VAE
- VQ-VAE
- VQ-GAN

Next week (week 9):

- Monday: Memorial day (no class)

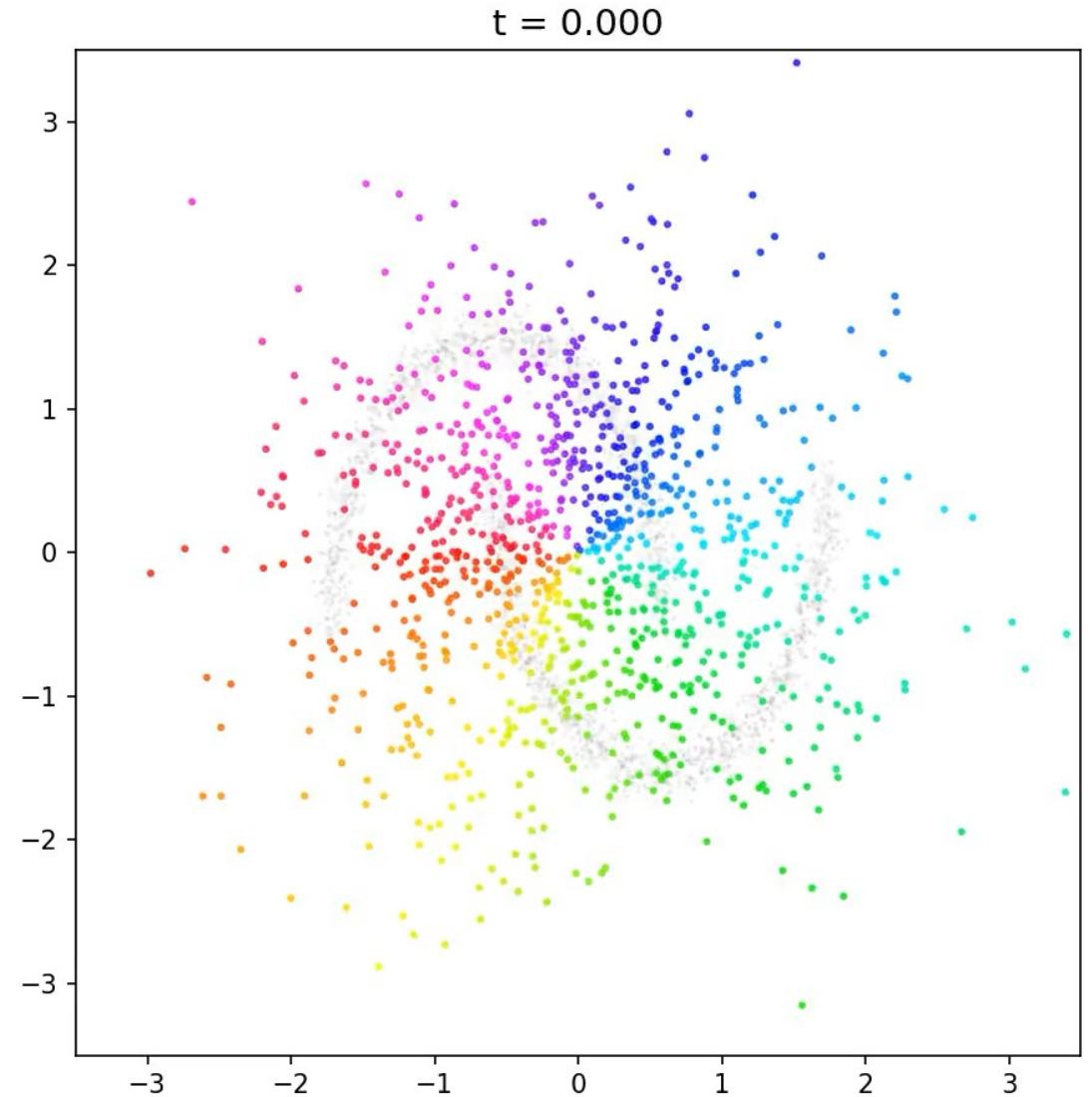
# Finish flow matching

With one movie and a healthy dose of math

I missed the best  
illustration in the demo

Start with  $z(t=0)$  from  
Gaussian (color points to  
make it easier to track  
where they go)

Simulate  $\dot{z} = \hat{v}(z, t)$



# Detail I missed from JIT!

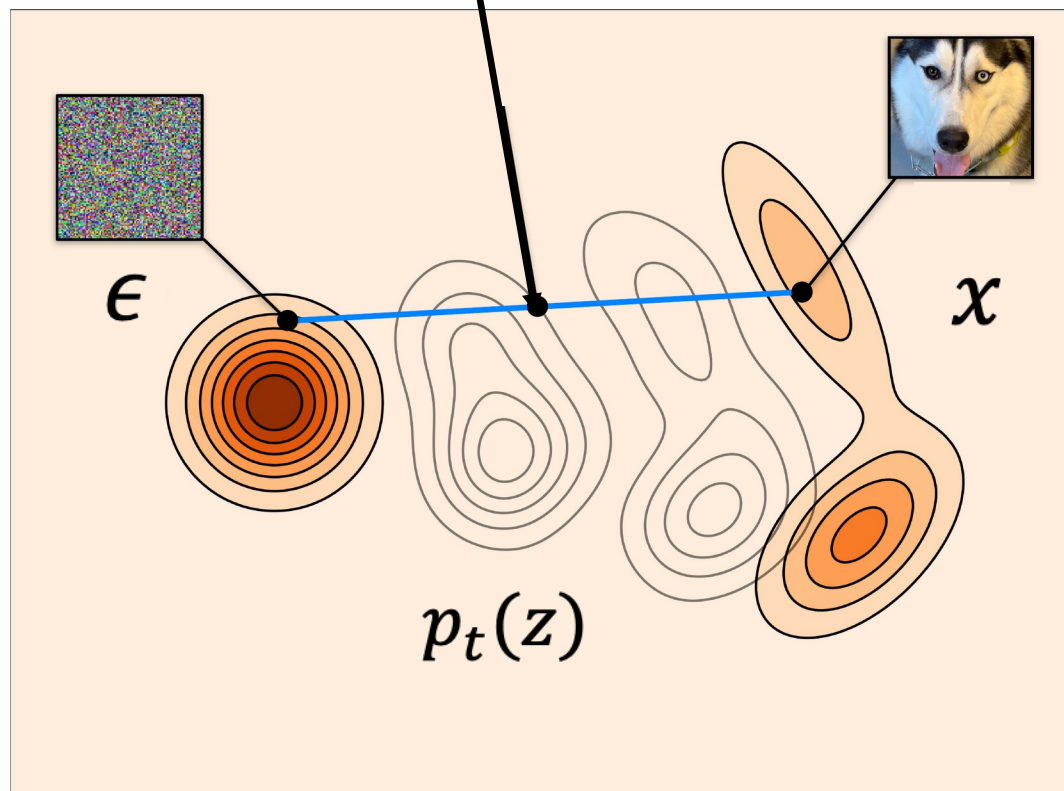
In HW 3 I'll suggest that:

- sample the logit normal during training, but
- constant linear time steps for sampling

|                               | JiT-B                                                                             | JiT-L | JiT-H | JiT-G |
|-------------------------------|-----------------------------------------------------------------------------------|-------|-------|-------|
| architecture                  |                                                                                   |       |       |       |
| depth                         | 12                                                                                | 24    | 32    | 40    |
| hidden dim                    | 768                                                                               | 1024  | 1280  | 1664  |
| heads                         | 12                                                                                | 16    | 16    | 16    |
| image size                    | 256 (other settings: 512, or 1024)                                                |       |       |       |
| patch size                    | image_size / 16                                                                   |       |       |       |
| bottleneck                    | 128 (B/L), 256 (H/G)                                                              |       |       |       |
| dropout                       | 0 (B/L), 0.2 (H/G)                                                                |       |       |       |
| in-context class tokens       | 32 (if used)                                                                      |       |       |       |
| in-context start block        | 4                                                                                 | 8     | 10    | 10    |
| training                      |                                                                                   |       |       |       |
| epochs                        | 200 (ablation), 600                                                               |       |       |       |
| warmup epochs [17]            | 5                                                                                 |       |       |       |
| optimizer                     | Adam [31], $\beta_1, \beta_2 = 0.9, 0.95$                                         |       |       |       |
| batch size                    | 1024                                                                              |       |       |       |
| learning rate                 | 2e-4                                                                              |       |       |       |
| learning rate schedule        | constant                                                                          |       |       |       |
| weight decay                  | 0                                                                                 |       |       |       |
| ema decay                     | {0.9996, 0.9998, 0.9999}                                                          |       |       |       |
| time sampler                  | $\text{logit}(t) \sim \mathcal{N}(\mu, \sigma^2)$ , $\mu = -0.8$ , $\sigma = 0.8$ |       |       |       |
| noise scale                   | $1.0 \times \text{image\_size} / 256$                                             |       |       |       |
| clip of $(1 - t)$ in division | 0.05                                                                              |       |       |       |
| class token drop (for CFG)    | 0.1                                                                               |       |       |       |
| sampling                      |                                                                                   |       |       |       |
| ODE solver                    | Heun [20]                                                                         |       |       |       |
| ODE steps                     | 50                                                                                |       |       |       |
| time steps                    | linear in [0.0, 1.0]                                                              |       |       |       |
| CFG scale sweep range [22]    | [1.0, 4.0]                                                                        |       |       |       |
| CFG interval [33]             | [0.1, 1] (if used)                                                                |       |       |       |



$$z_t = (1 - t)\epsilon + t x$$



$$\dot{z}_t = \underbrace{x - \epsilon}_v$$

Train a neural network to predict “v”

Let’s show that this works – i.e. using:

$$\dot{z}_t = \hat{v}(z, t)$$

Gives the same distributions!  
(for optimal  $\hat{v}$ )

# Loss

$$\mathbb{E}_{t, \epsilon, x} [\|\hat{v}(z, t) - v\|^2]$$

Derive the “unconditional” flow  $z_t = (1-t)\epsilon + t x$

$$\dot{z}_t = \mathbf{v} = x - \epsilon = \frac{x - z}{1 - t}$$

Use “Continuity equation” to get dynamics of  $p_t(z|x)$

$$\partial_t p_t(z|x) = -\nabla \cdot (p_t(z|x) \mathbf{v}) = -\nabla \cdot (p_t(z|x) (x - z)/(1 - t))$$

Take expectation over  $p(x)$

$$\begin{aligned} \int dx p(x) \partial_t p_t(z|x) &= \partial_t p_t(z) \\ &= \int dx p(x) \left[ -\nabla \cdot \left( p_t(z|x) \frac{(x - z)}{(1 - t)} \right) \right] \\ &= -\nabla \cdot (p_t(z) \frac{(\mathbb{E}[x|z, t] - z)}{(1 - t)}) \end{aligned}$$

# Conditional mean minimizes mean square error

$$\arg \min_{\hat{x}(z,t)} \mathbb{E}_{p_t(z|x)p(x)}[|x - \hat{x}(z,t)|^2]$$

Then:

$$\hat{x}(z,t) = \mathbb{E}_{p_t(x|z)}[x] = \mathbb{E}[x|z,t]$$

Conclusion: the min MSE denoiser gives the correct “flow”

# Tweedie's formula

$$\nabla_z \log p_t(z) = \frac{t}{(1-t)^2} \mathbb{E}[x|z, t] - \frac{z}{(1-t)}$$

The expression on the left is called the “score”.  
Shows up everywhere.

**Score is related to optimal denoiser!!**

*Higher order Tweedie identities: Dytso et al “Meta Derivative Identity for the Conditional Expectation”*



# Diffusion – reverse SDE dynamics

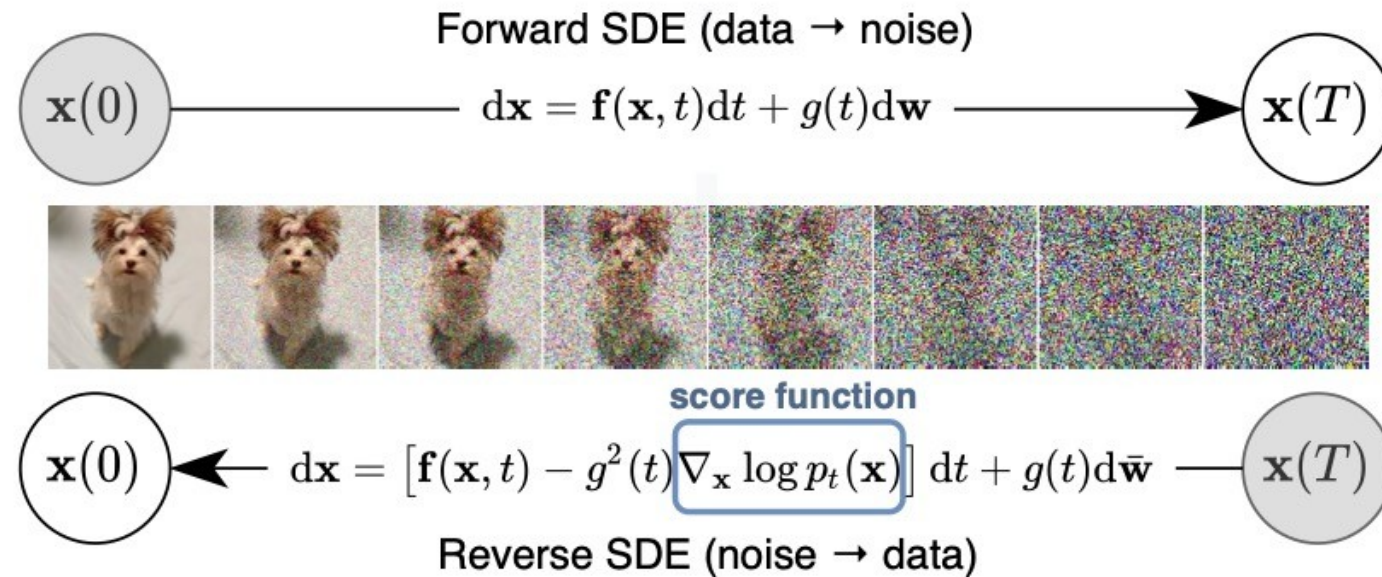


Figure 1: **Solving a reverse-time SDE yields a score-based generative model.** Transforming data to a simple noise distribution can be accomplished with a continuous-time SDE. This SDE can be reversed if we know the score of the distribution at each intermediate time step,  $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ .

Famous paper:

**Score-Based Generative Modeling through Stochastic Differential Equations**

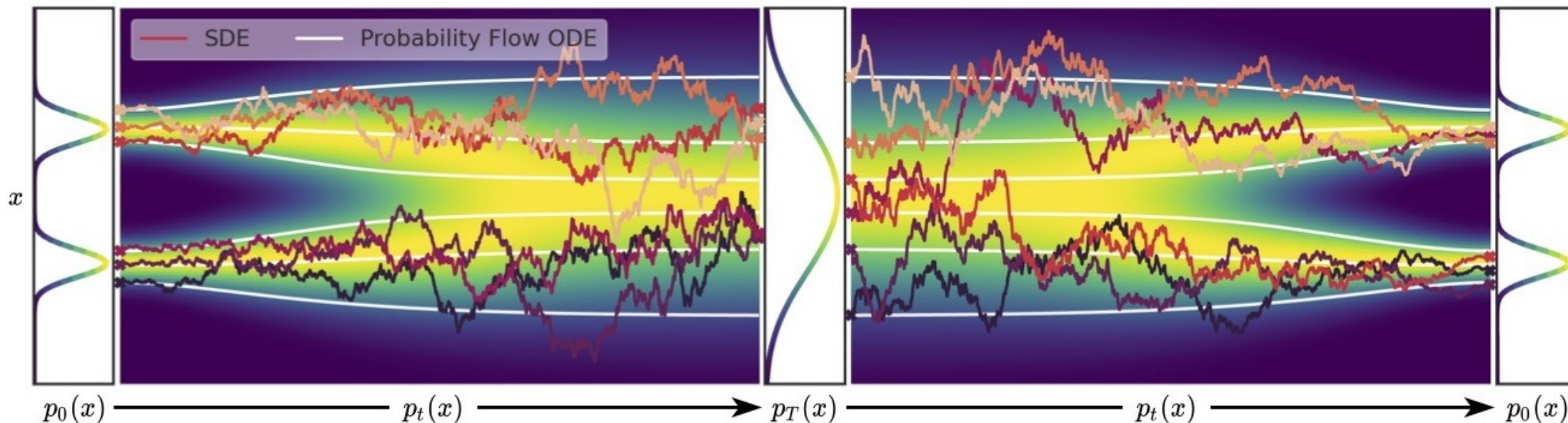
# Convert to continuous flow?

This ODE has the same marginal densities. (Fokker Planck again!)  
 So if we replace the learned score function, we can solve this ODE to get probability density estimates, with the

$$d\mathbf{x} = \left[ \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt.$$

change of variables formula

$$\text{Data } x(0) \xrightarrow{dx = f(x,t)dt + g(t)dw} \text{Prior } x(T) \xrightarrow{dx = [f(x,t) - g^2(t)\nabla_x \log p_t(x)]dt + g(t)d\bar{w}} \text{Reverse SDE} \xrightarrow{\text{Data}} x(0)$$



---

# Elucidating the Design Space of Diffusion-Based Generative Models

---

Tero Karras  
NVIDIA

Miika Aittala  
NVIDIA

Timo Aila  
NVIDIA

Samuli Laine  
NVIDIA

## Abstract

We argue that the theory and practice of diffusion-based generative models are currently unnecessarily convoluted and seek to remedy the situation by presenting a design space that clearly separates the concrete design choices. This lets us identify several changes to both the sampling and training processes, as well as preconditioning of the score networks. Together, our improvements yield new state-of-the-art FID of 1.79 for CIFAR-10 in a class-conditional setting and 1.97 in an unconditional setting, with much faster sampling (35 network evaluations per image) than prior designs. To further demonstrate their modular nature, we show that our design changes dramatically improve both the efficiency and quality obtainable with pre-trained score networks from previous work, including improving the FID of a previously trained ImageNet-64 model from 2.07 to near-SOTA 1.55, and after re-training with our proposed improvements to a new SOTA of 1.36.

Continuous normalizing flow –  
just needs the score function

$$d\mathbf{x} = -\dot{\sigma}(t) \sigma(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma(t)) dt,$$

Then Fokker Planck equation  
says that  
 $p(\mathbf{x}; \sigma(t))$  evolves from  
Gaussian to target distribution  
under these dynamics!

• How do we get  $\nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma(t))$

• It happens to be MSE!

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \propto -\hat{\epsilon}(\mathbf{x}, t)$$

Where the denoiser optimizes:

$$\min_{\hat{\epsilon}} E[|\epsilon - \hat{\epsilon}(\mathbf{x}, t)|^2]$$

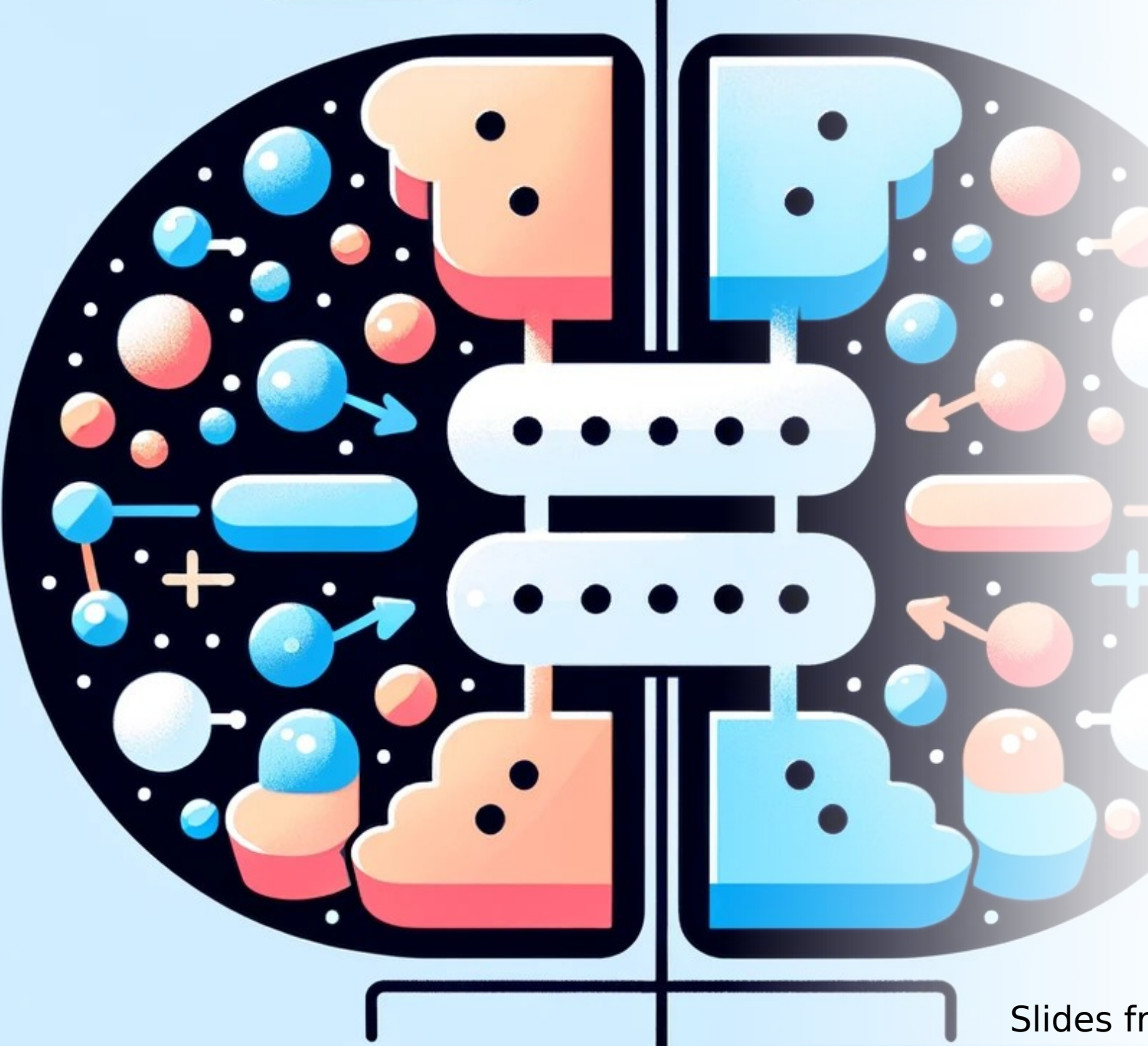
The same kind of MSE objective we get from VAE and flow matching perspective! (but maybe weighted differently)

Continuous normalizing flow – just needs the score function

$$d\mathbf{x} = -\dot{\sigma}(t) \sigma(t) \nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma(t)) dt,$$

Then Fokker Planck equation says that  $p(\mathbf{x}; \sigma(t))$  evolves from Gaussian to target distribution under these dynamics!

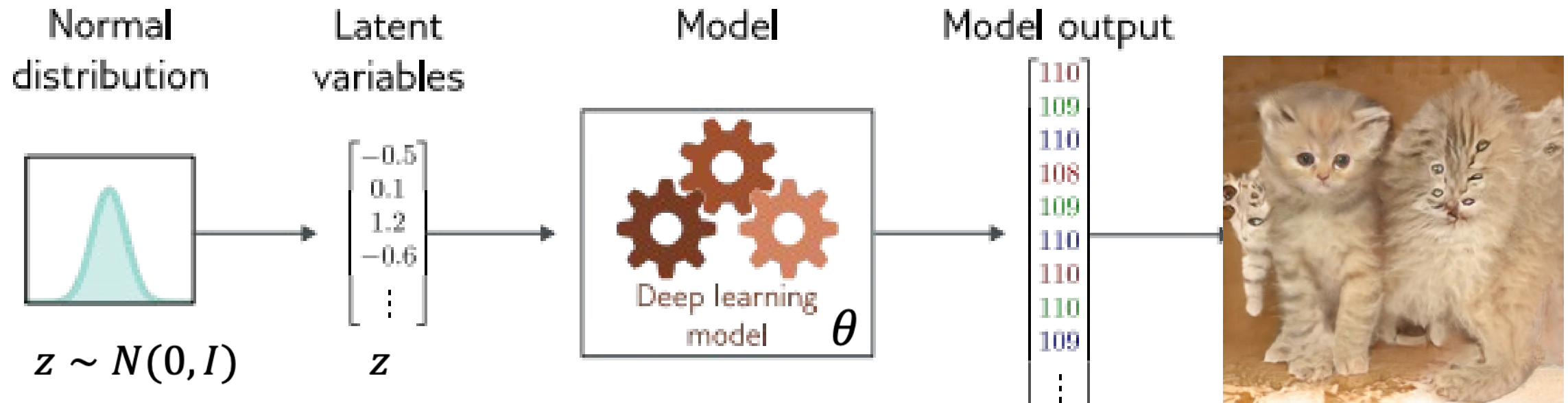




# Generative Adversarial Networks (GANs)

A few unique ideas in ML, and new work tries to combine GANs with Diffusion, or uses ideas from GANs in new context (like CycleGAN)

# Latent variable models (VAE or GAN)



Latent variable models map a random “latent” variable to create a new data sample

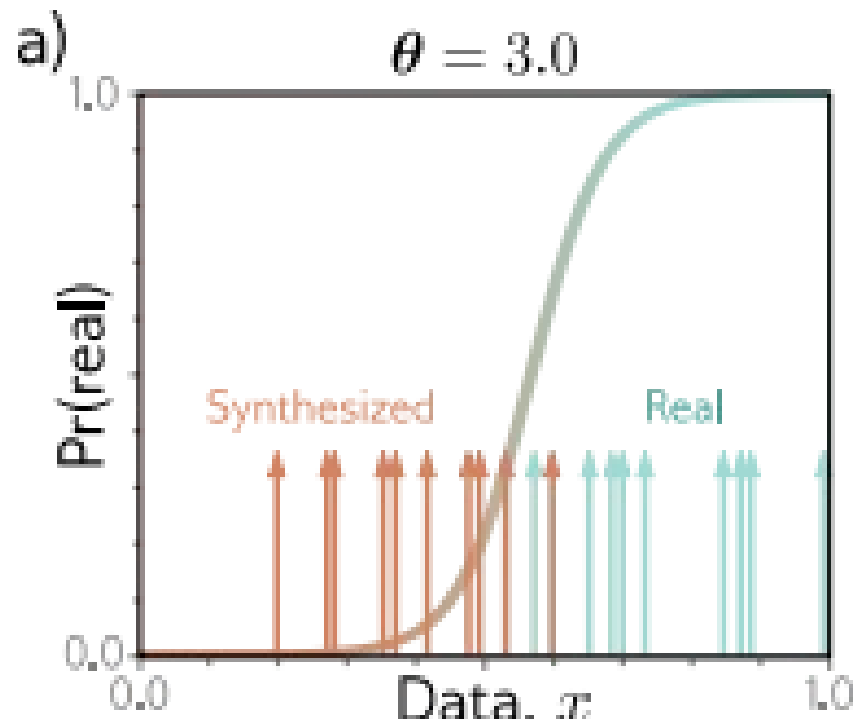
In GANs, this is called the **generator**:

$$\mathbf{x}_* = \mathbf{g}[\mathbf{z}_i, \theta]$$



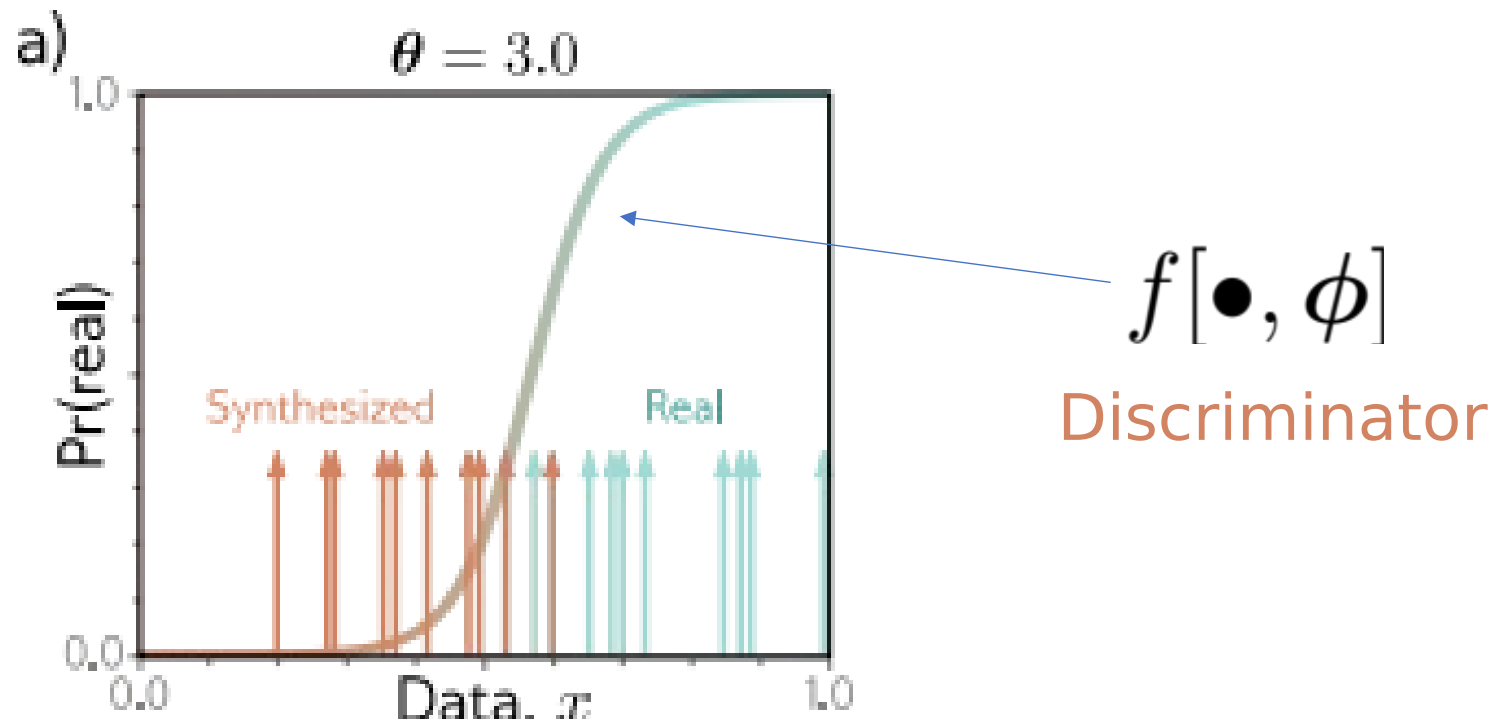
# GAN example

$$x_j^* = g[z_j, \theta] = z_j + \theta$$



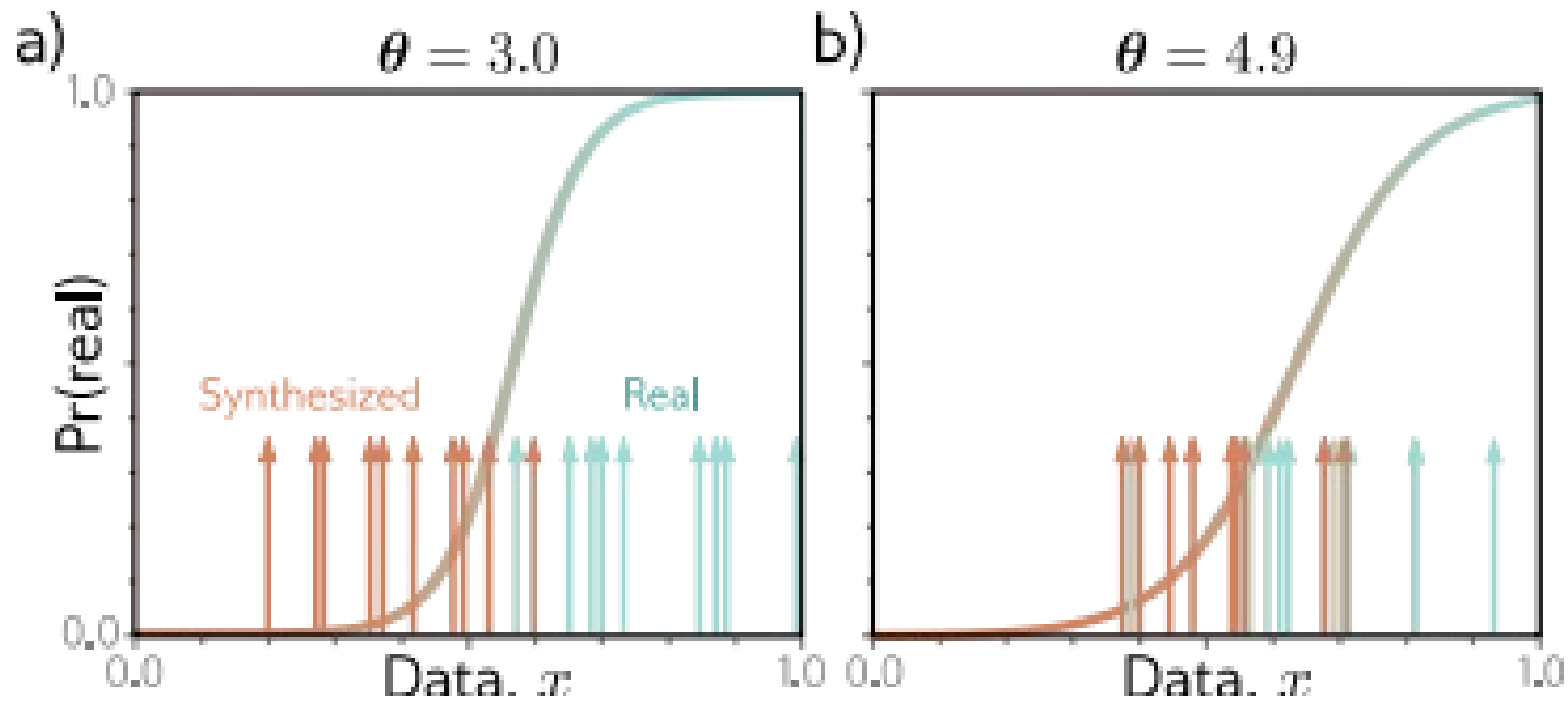
# GAN example

$$x_j^* = g[z_j, \theta] = z_j + \theta$$



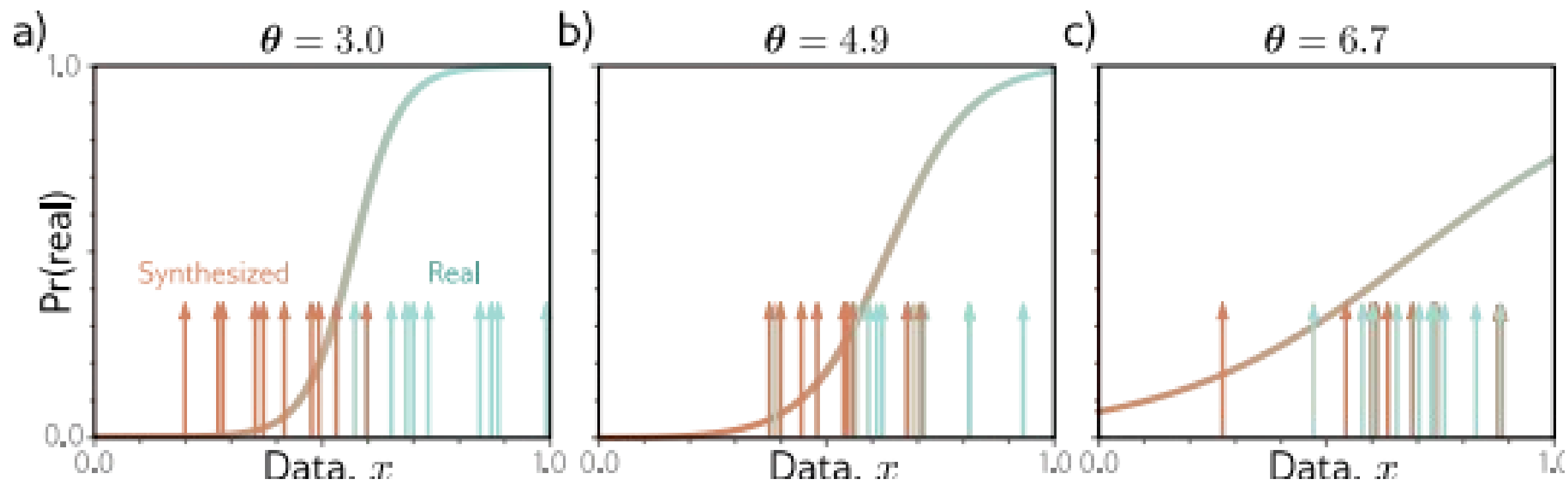
# GAN example

$$x_j^* = g[z_j, \theta] = z_j + \theta$$



# GAN example

$$x_j^* = g[z_j, \theta] = z_j + \theta$$



# GAN cost function

Discriminator uses standard cross entropy loss:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_i -(1 - y_i) \log [1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$



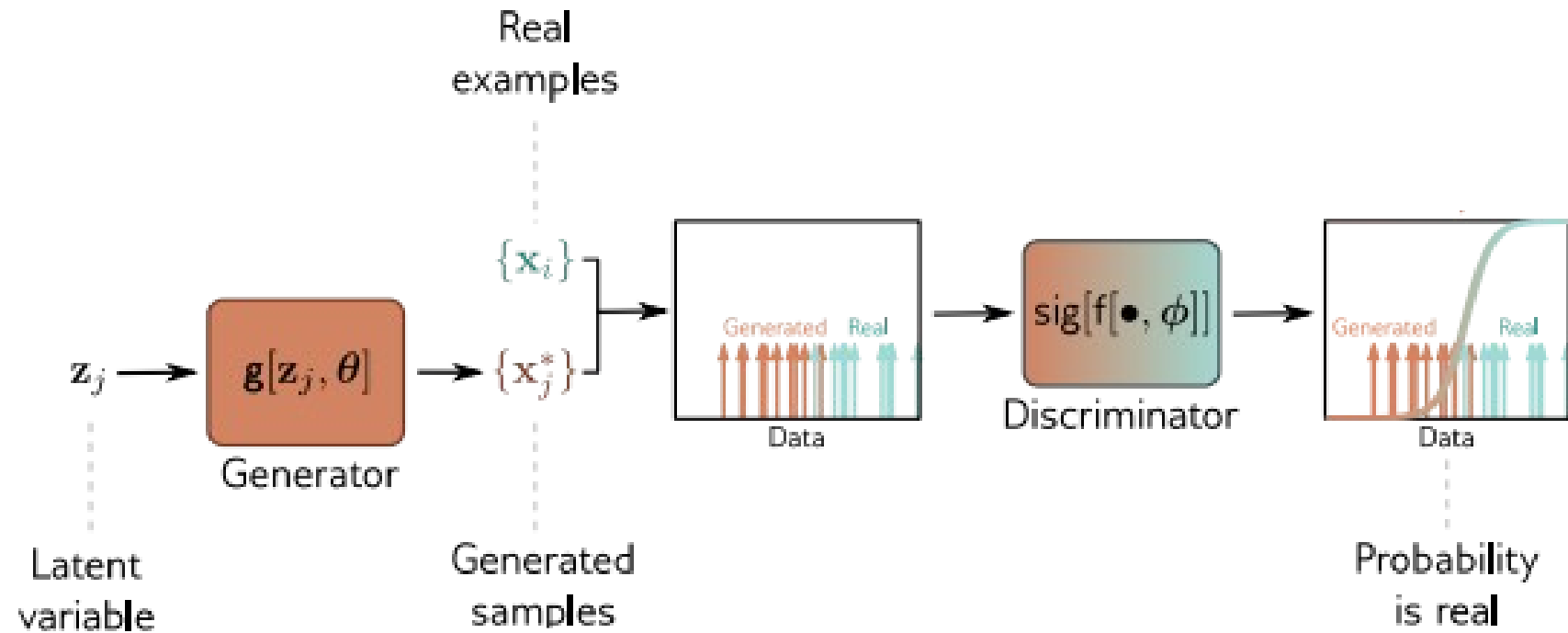
$x_i$  ,  $y_i = 0$  FAKE



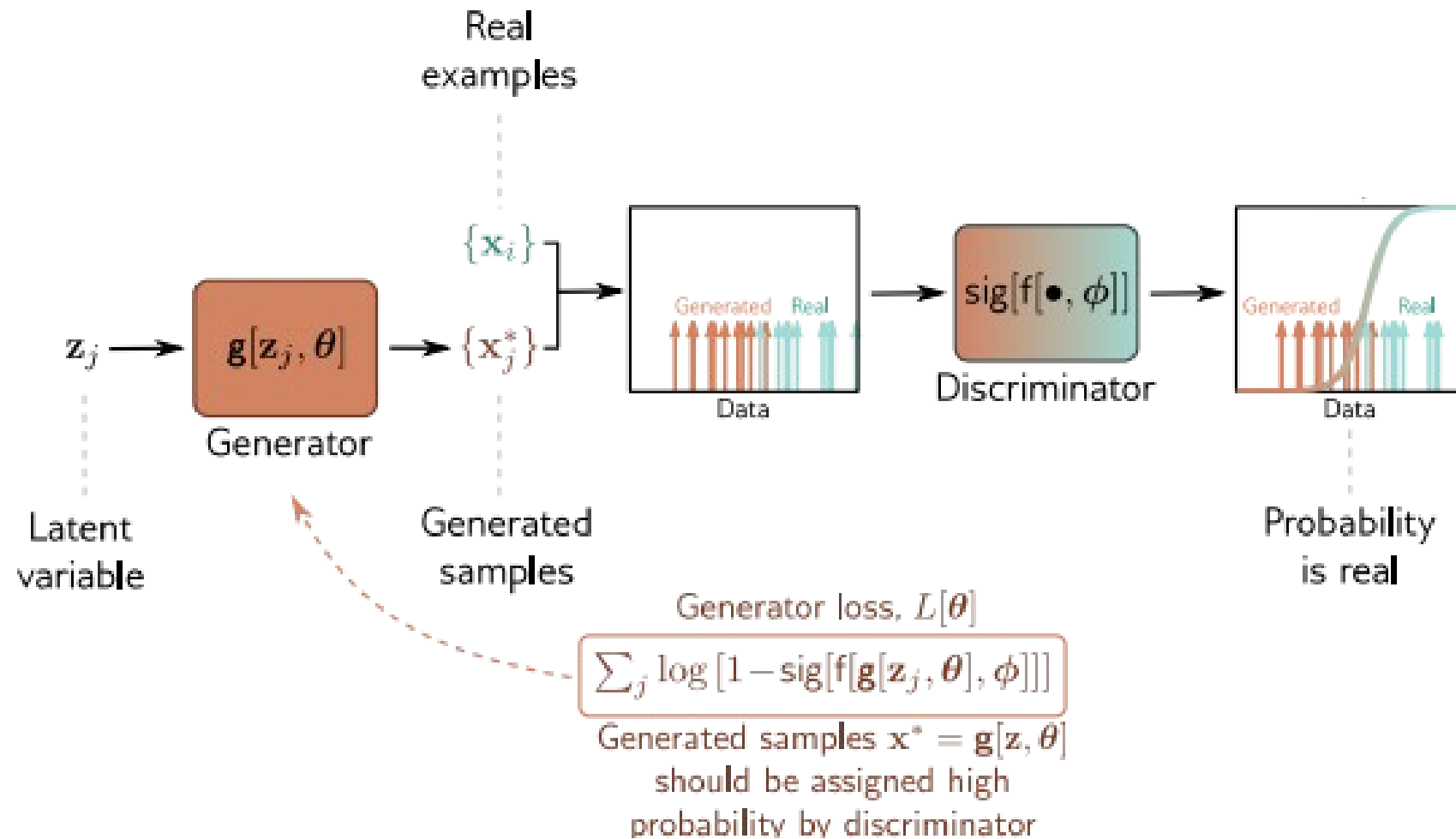
$x_i$  ,  $y_i = 1$  REAL



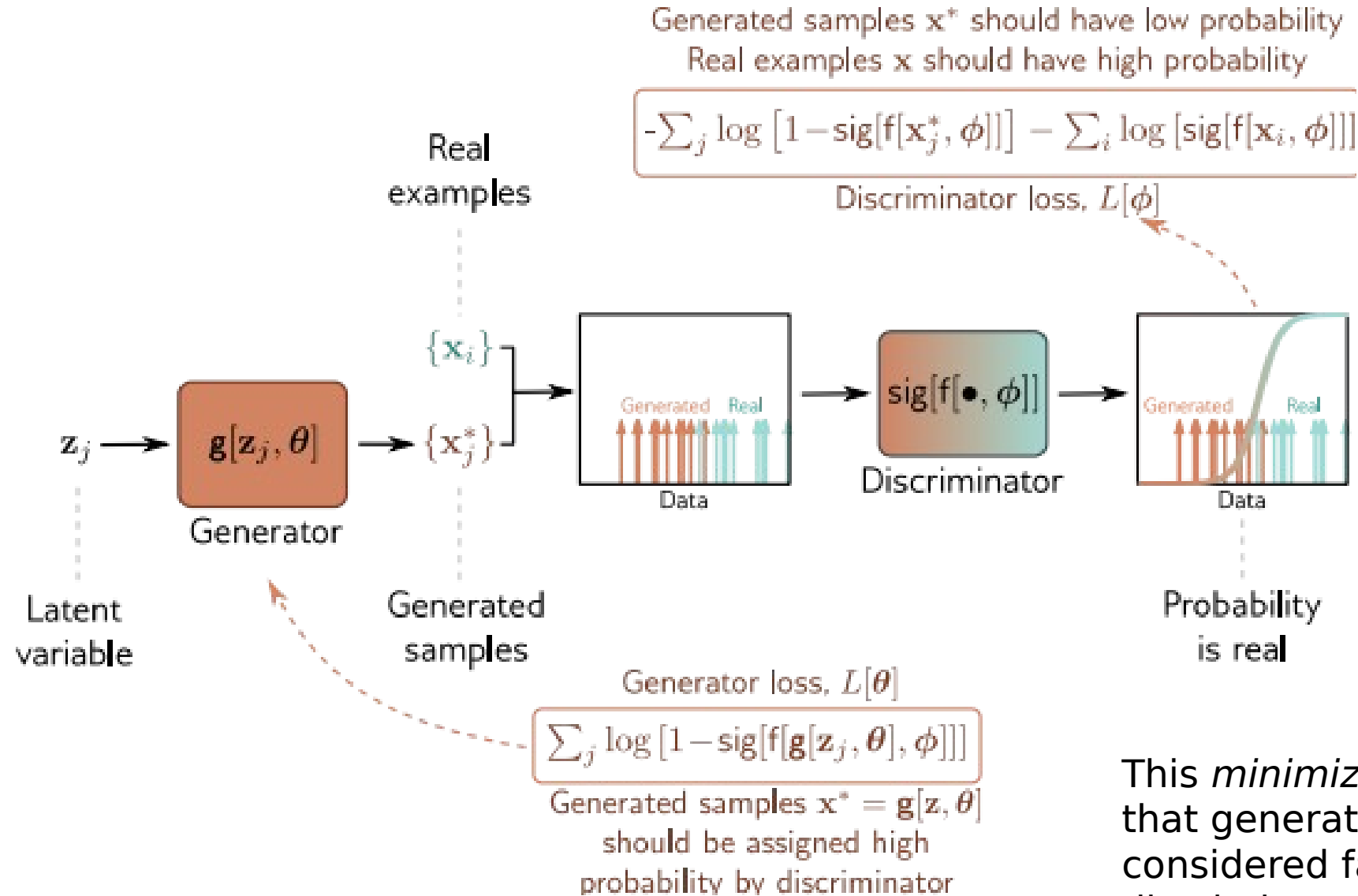
# GAN loss function



# GAN loss function



# GAN loss function



This *minimizes* probability that generated image is considered fake, by discriminator

# GAN cost function

Discriminator uses standard cross entropy loss:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_i -(1 - y_i) \log [1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

Discriminator: generated samples,  $y = 0$ , real examples,  $y = 1$ :

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_j -\log [1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

# GAN cost function

Discriminator uses standard cross entropy loss:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_i -(1 - y_i) \log [1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

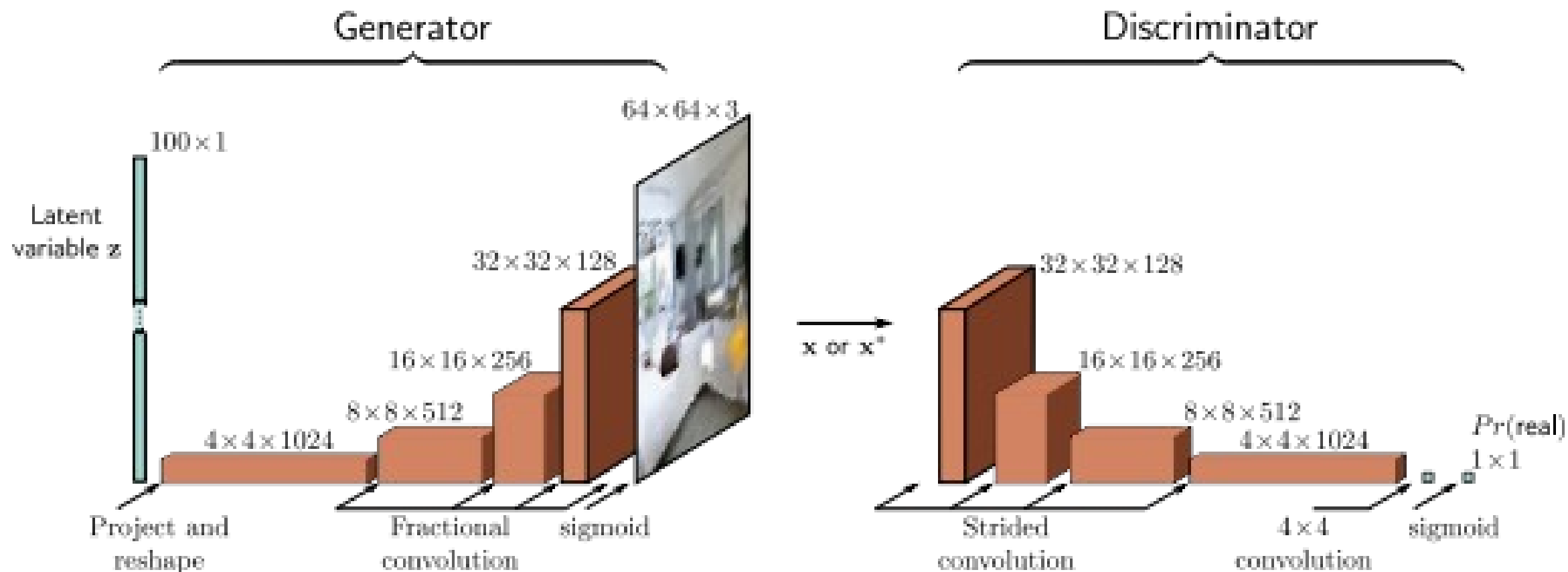
Discriminator: generated samples,  $y = 0$ , real examples,  $y = 1$ :

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_j -\log [1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

Generator loss: make generated samples more likely under discriminator (i.e. make discriminator loss larger)

$$\hat{\phi}, \hat{\theta} = \operatorname{argmin}_{\phi} \left[ \operatorname{argmax}_{\theta} \left[ \sum_j -\log [1 - \operatorname{sig}[f[\mathbf{g}[\mathbf{z}_j, \theta], \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right] \right]$$

# Deep Convolutional (DC) GAN



# DC GAN Results

a)



b)



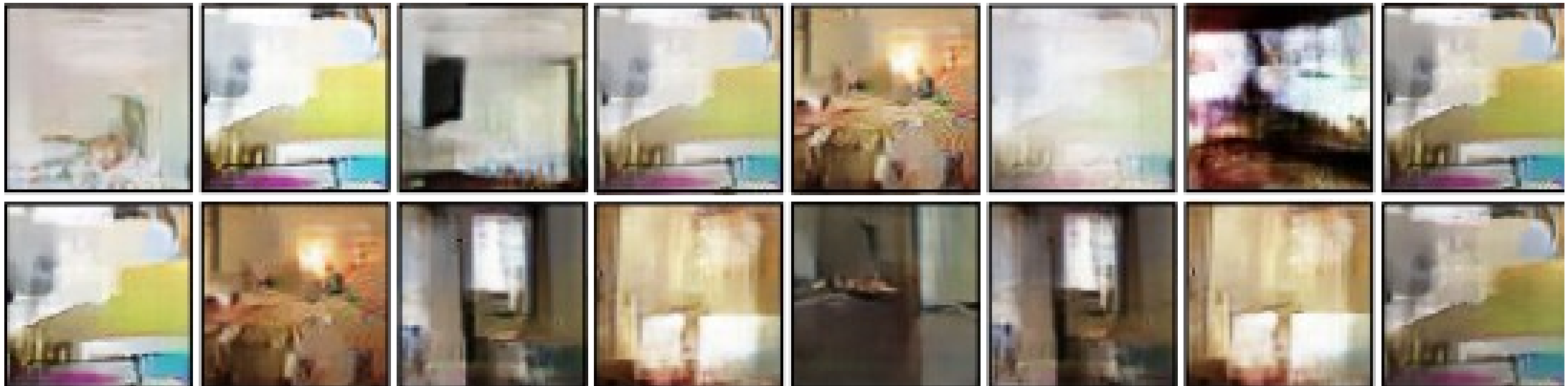
c)



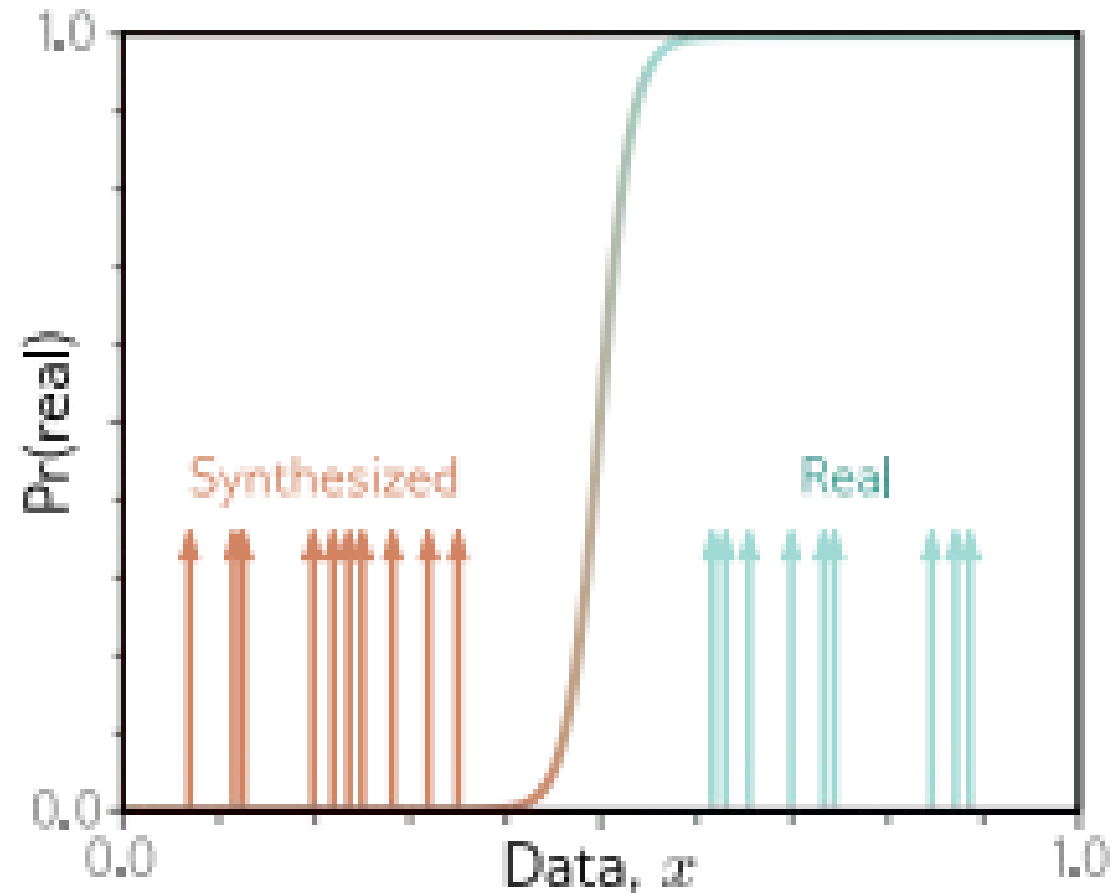


# “Mode collapse”

Training GANs is difficult. Often fails.



# GAN training instability



# Solve with Wasserstein GAN loss function?

- Original function:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[ \sum_j -\log \left[ 1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]] \right] - \sum_i \log \left[ \operatorname{sig}[f[\mathbf{x}_i, \phi]] \right] \right]$$

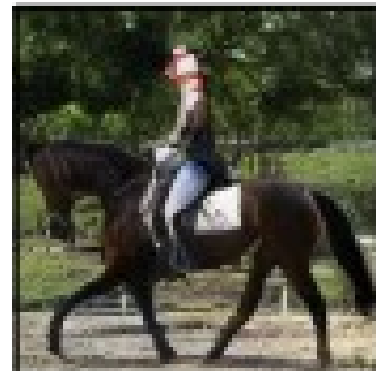
- Wasserstein GAN:

$$L[\phi] = \sum_j f[\mathbf{x}_j^*, \phi] - \sum_i f[\mathbf{x}_i, \phi]$$

subject to:

$$\left| \frac{\partial f[\mathbf{x}, \phi]}{\partial \mathbf{x}} \right| < 1$$

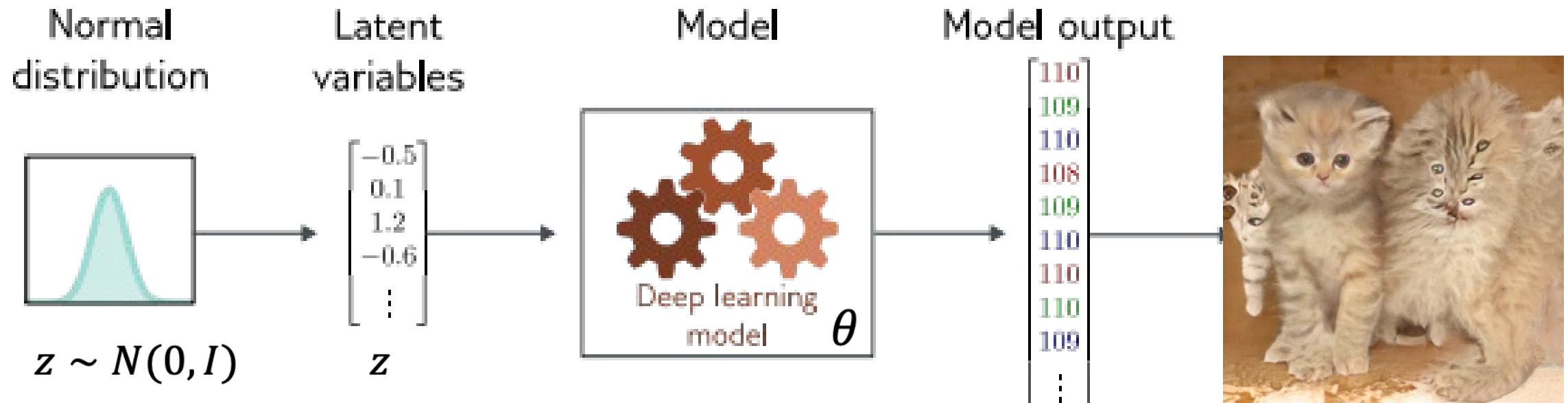
# Example results



# More generative model tricks

Most of these first illustrated with GANs, but now also done using other methods

# Latent variable models (VAE or GAN)



Latent variable models map a random “latent” variable to create a new data sample

In GANs, this is called the **generator**:

$$\mathbf{x}_* = \mathbf{g}[\mathbf{z}_i, \theta]$$

# Interpolation (in latent space)



A common follow-up: Can we design so that certain dimensions control certain qualities?

# Image Translation

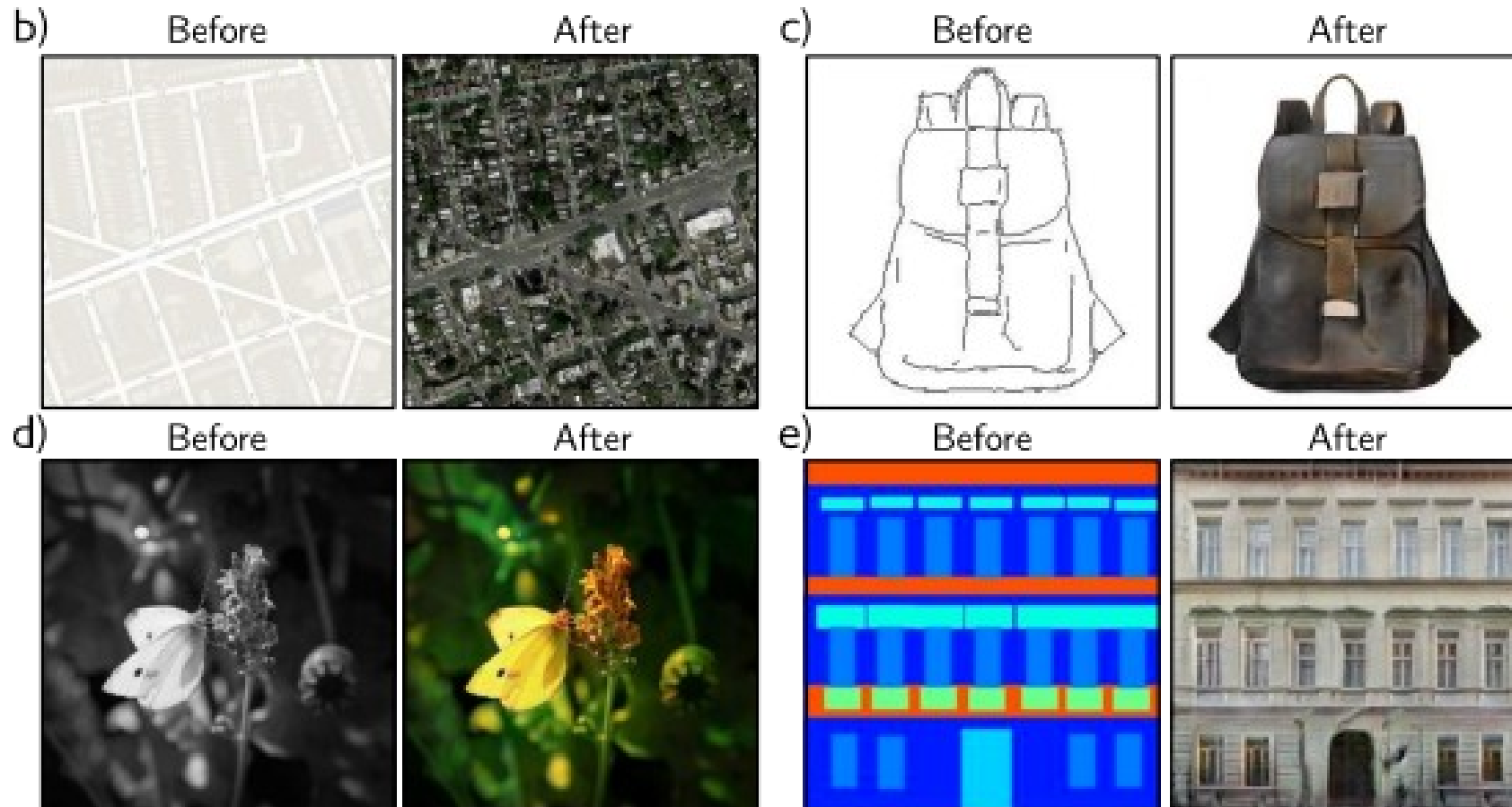
Methods:

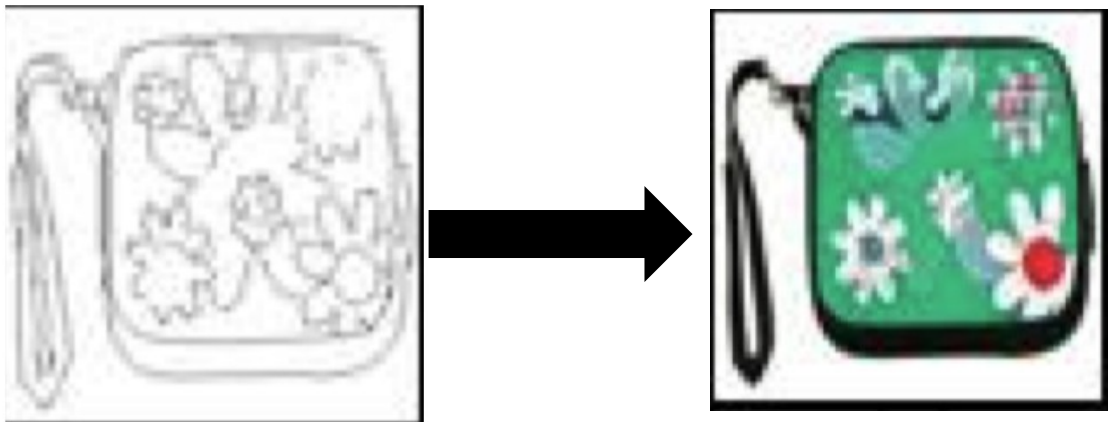
- ControlNet + diffusion
- Diffusion bridges
- GANs
- Regular supervised learning
- ?



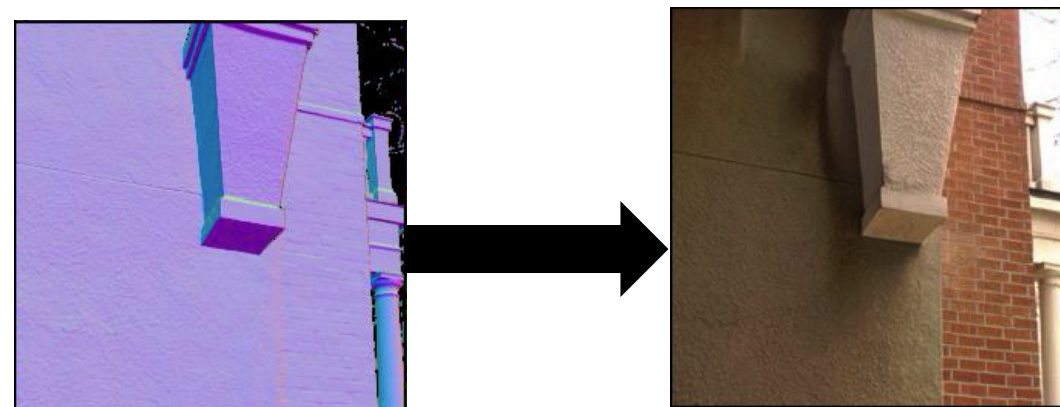
# Image translation: Pix2Pix

“paired image-to-image translation”

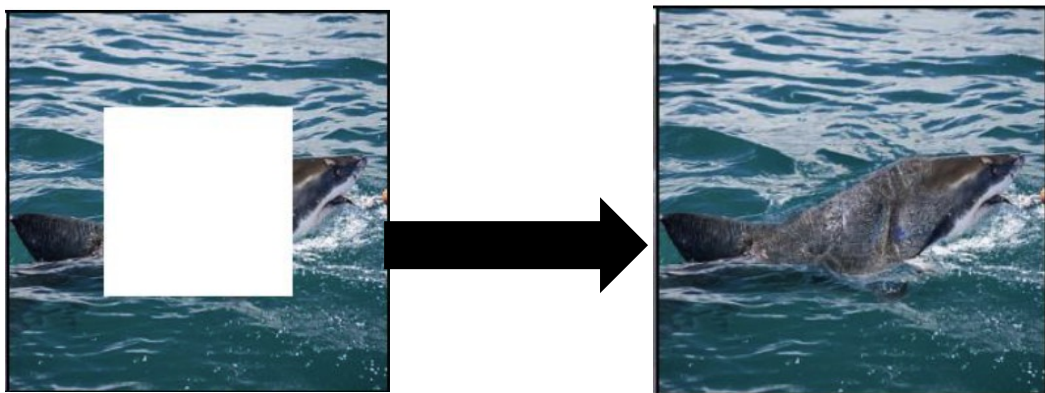




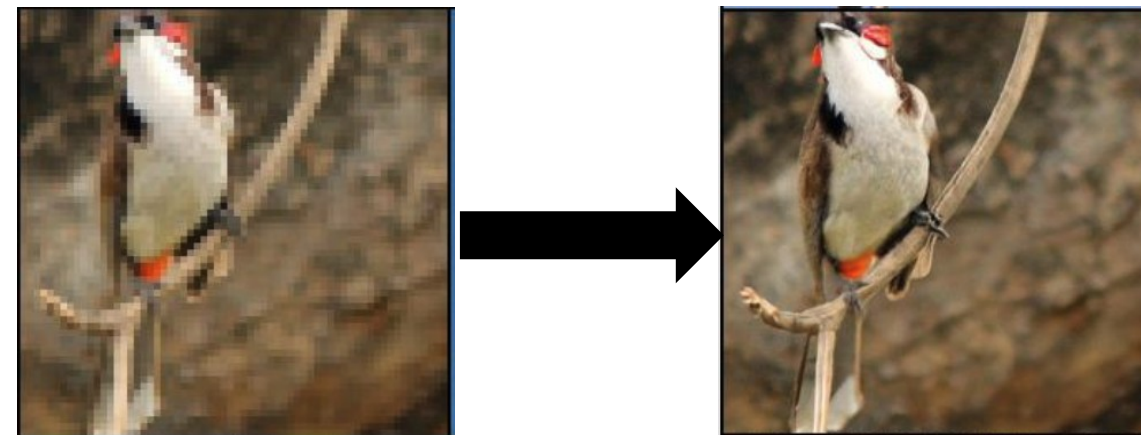
Edges to handbags



**Depth to RGB images**

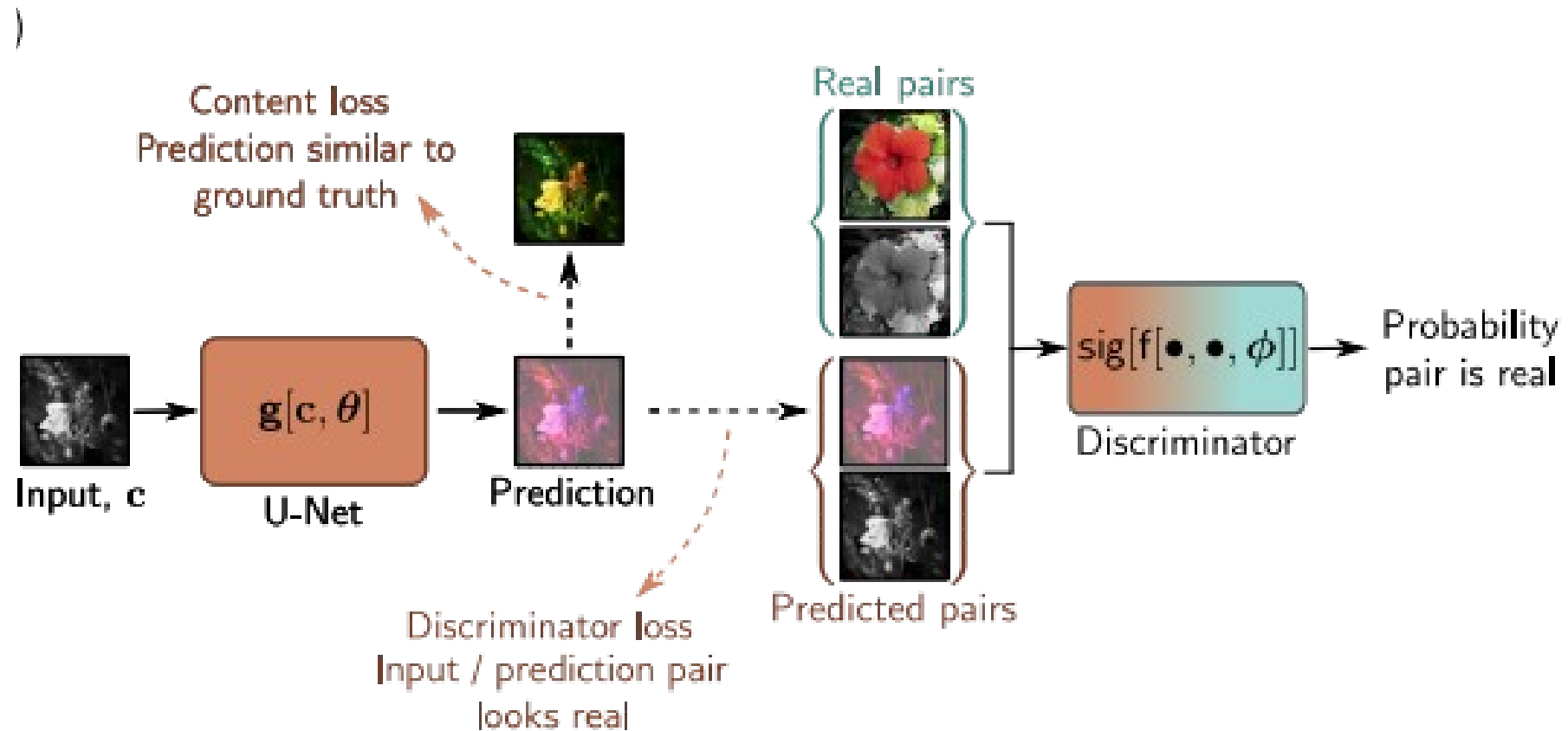


**Inpainting**



Super-resolution

# Image translation GAN style: Pix2Pix



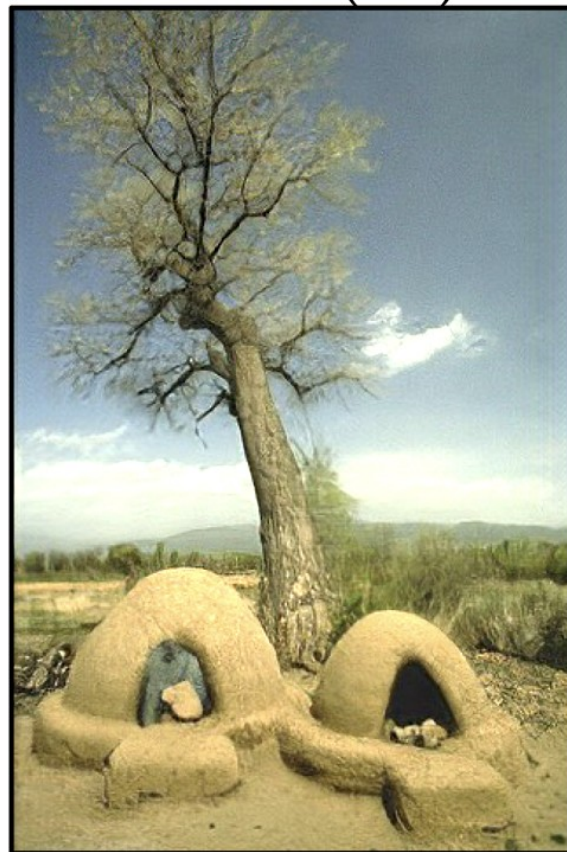


# Image translation example: Super-resolution

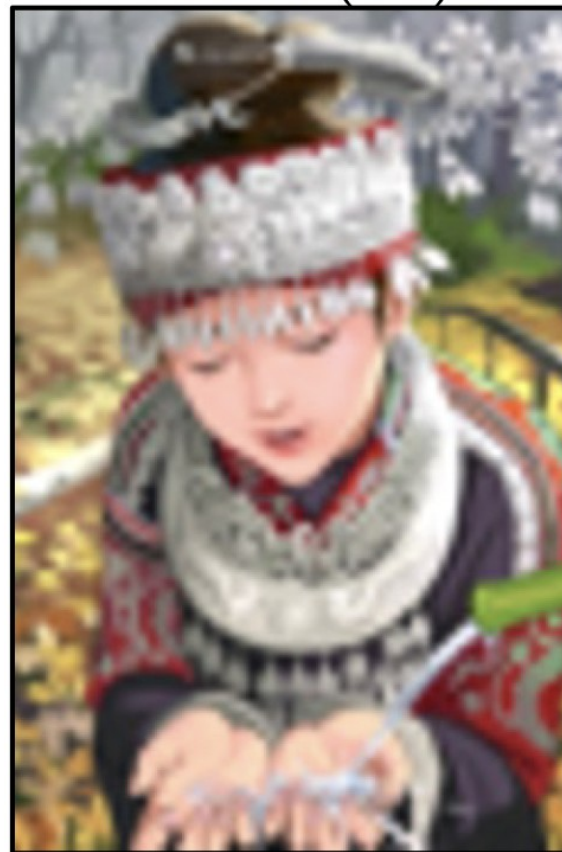
b) Bicubic (4 $\times$ )



c) SRGAN (4 $\times$ )



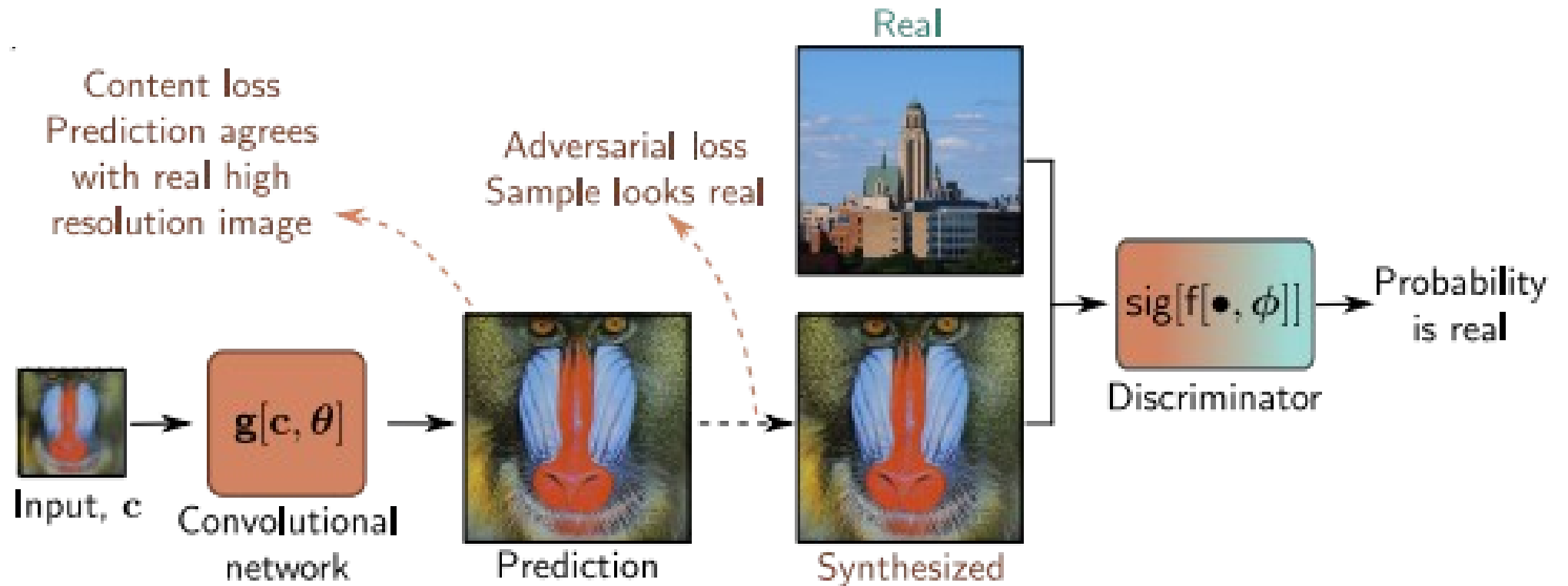
d) Bicubic (4 $\times$ )



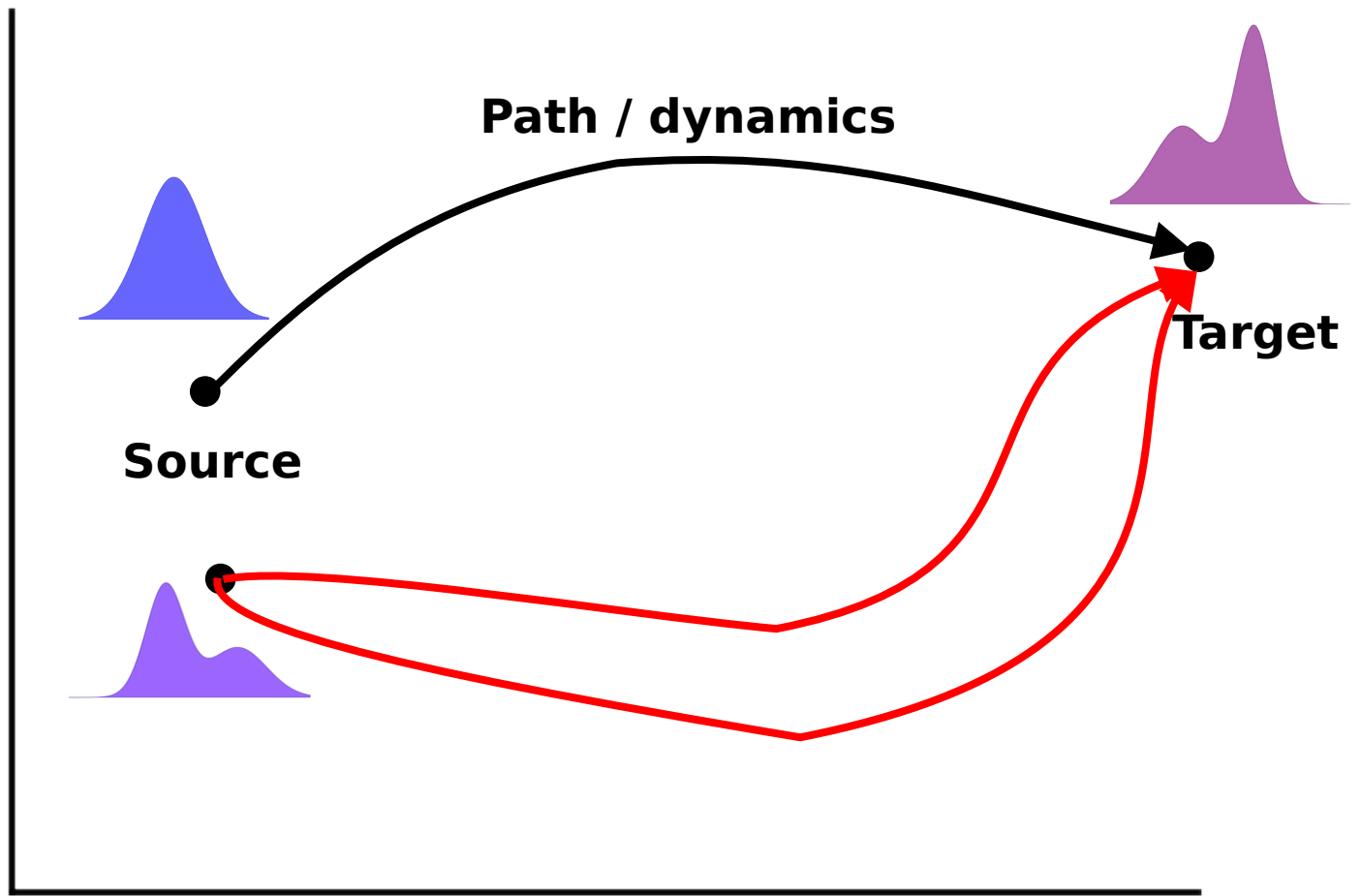
e) SRGAN (4 $\times$ )



# Image translation: Super Resolution GAN



# Diffusion bridges



**DDBM.** Zhou L, Lou A, Khanna S, et al. Denoising Diffusion Bridge Models[C]//The Twelfth International Conference on Learning Representations.

**DBIM.** Zheng K, He G, Chen J, et al. Diffusion bridge implicit models[J]. arXiv preprint arXiv:2405.15885, 2024.

**DSBM.** Shi Y, De Bortoli V, Campbell A, et al. Diffusion schrödinger bridge matching[J]. Advances in Neural Information Processing Systems, 2023, 36: 62183-62223.

**SI.** Albergo M S, Boffi N M, Vanden-Eijnden E. Stochastic interpolants: A unifying framework for flows and diffusions[J]. arXiv preprint arXiv:2303.08797, 2023.

Shaorong Zhang, .. Ver Steeg group: NeurIPS 2025, Exploring the Design Space of Diffusion Bridge Models via Stochasticity Control

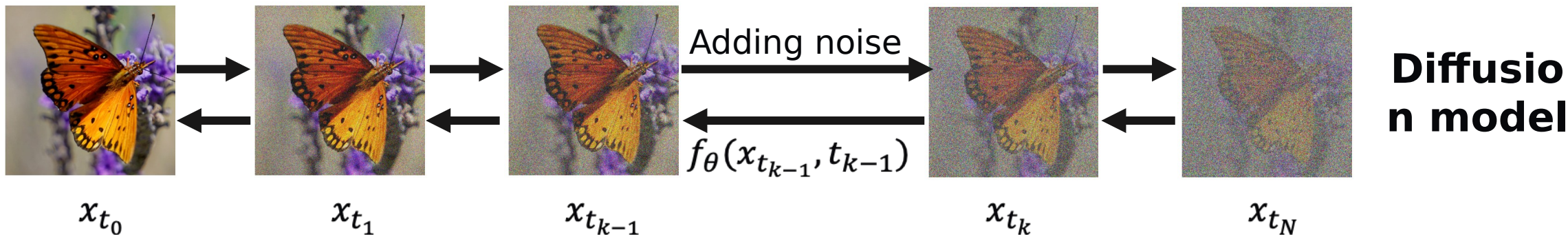




# Diffusion versus Diffusion Bridges

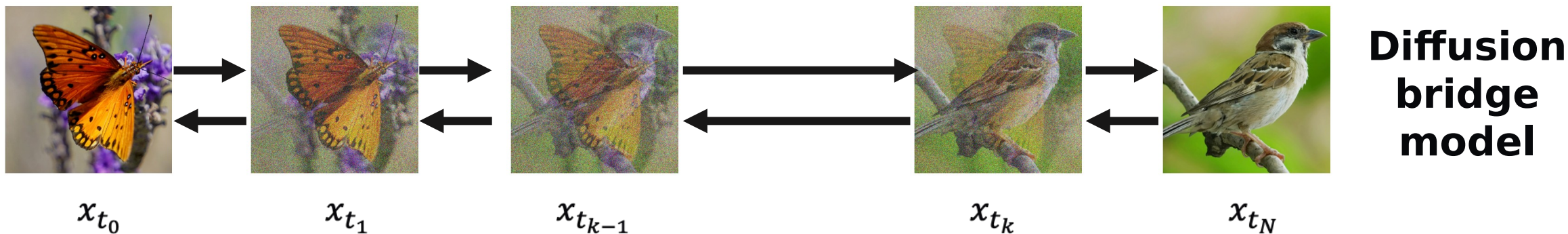


Transport between Gaussian and target distribution



A special case

Transport between any two distributions



# Stochastic control for better *conditional diversity* in sampling

Ours



DDBM

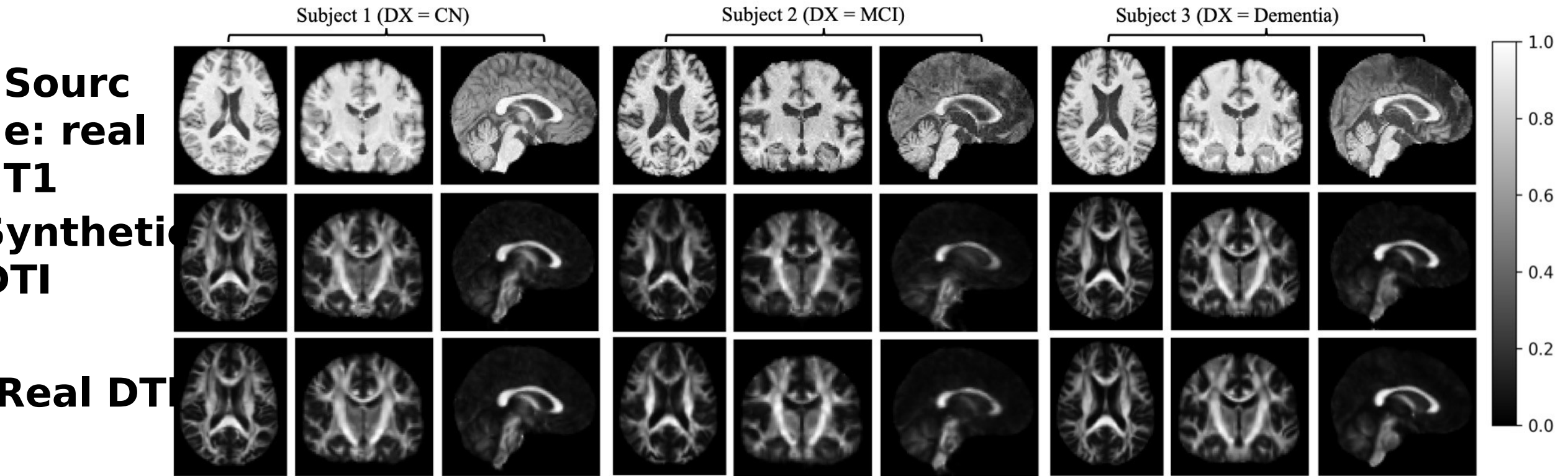


Measuring conditional  
diversity

| Model   | Sampler | NFEs ↓   | FID ↓       | AFD ↑       |
|---------|---------|----------|-------------|-------------|
| DDBM-VP | DDBM    | 118      | 1.83        | 6.99        |
| DDBM-VP | SDB     | 40       | 1.16        | 6.29        |
| Ours    | SDB     | <b>5</b> | <b>0.89</b> | 6.00        |
| Ours    | SDB     | 40       | 1.71        | <b>9.52</b> |



# Medical Image Translation



- Visually, very good recovery of DTI
- Downstream task performance (Alzheimer's classification) almost as good! (90%, synthesized VS 91% real)

# “unpaired image-to-image translation”

Pictures of horses



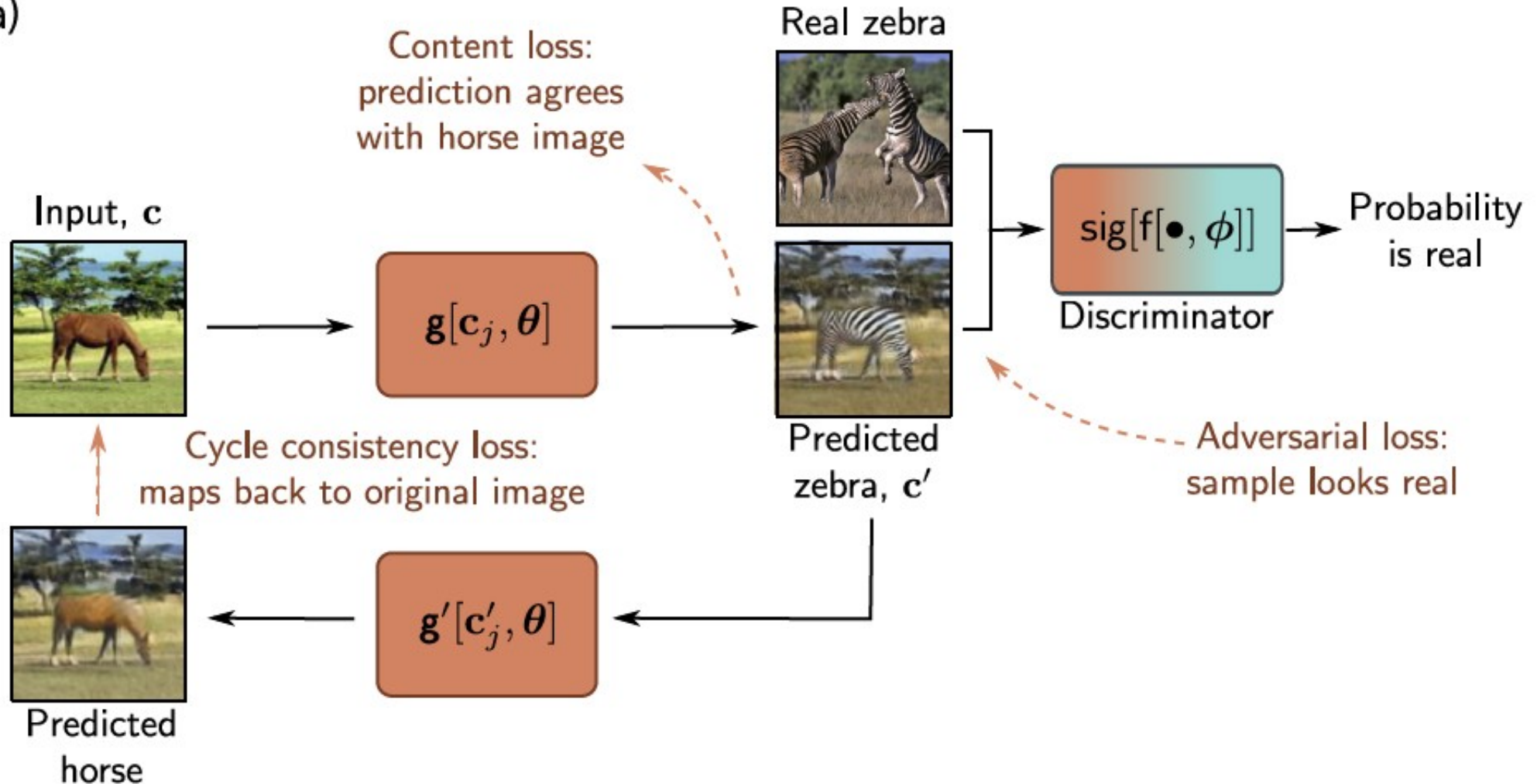
(Unrelated) Pictures of zebras



Without changing *any other details of the image*, change this horse to a zebra

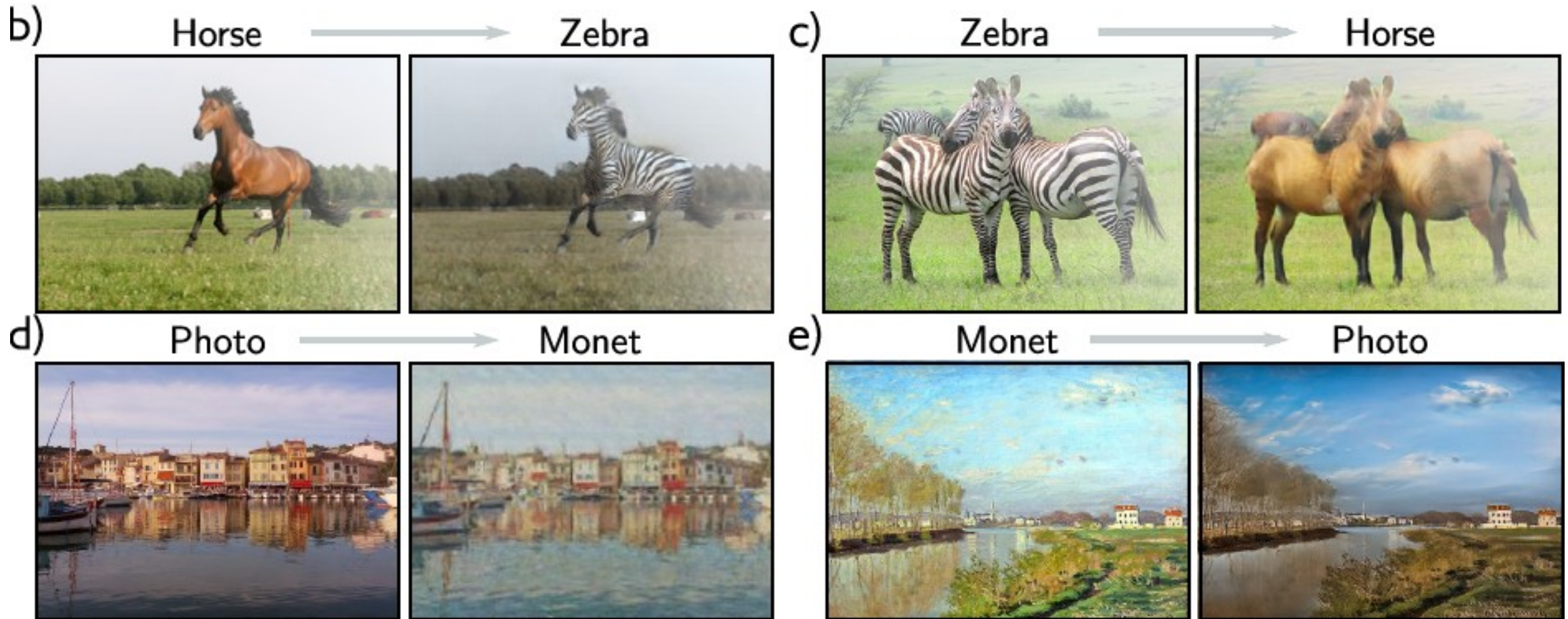
# Image translation: CycleGAN

a)





# Image translation: CycleGAN



# Example limitations from original



<https://github.com/junyanz/CycleGAN>

WEDNESDAY:  
Variational Auto-Encoder  
(VAE)  
And VQ-VAE, VQ-GAN

# Vector Quantized: VQ-GAN

<https://compvis.github.io/taming-transformers>



